

ANIMESH MUKHERJEE (AM)

Discrete Structures.

Propositional Logic
Predicate Logic
Proof techniques

Logical Reasoning

Sets & their properties

- Sets
- Relations
- Partial orders
- Bounds
- Size of sets.

Permutation/Combination
Recurrence Relation
Generating functions

Counting

Algebraic structures

- Groups
- Rings / Fields
- Graphs
- Matrices.

Task of a Computer Scientist?

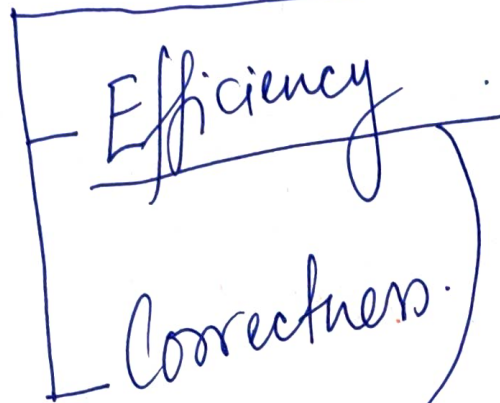
Building Machines

Using Machines.

X [Hardware (Digital Circuits
Computer Architecture).

X [using browsers?
Using OpenAI /
Anthropic playgrounds
foundations of
problem solving.
(computational problem solving).

Problem Solving → Designing Algorithms.



Given an algorithm make it as efficient as possible.

Efficiency Analysis →

number of operations in terms of the input size (n).

Counting problem formulation helps in efficiency analysis.

Correctness — proposing an algorithm
proof techniques. → it should work on all inputs → proved.

Thus:

Counting leads toward efficiency analysis
Logic + proof techniques leads toward correctness.

Logic Reasoning.

Logic — [— propositional logic
— Predicate logic.

Proof technique — [— Direct proof
— Proof by contradiction
— Proof by contraposition
— Proof by induction.

Setup: Let A & B be two non-empty arrays of positive integers (repetitions allowed).

Definition. $A \leq B \iff$ for every $a \in A$ there exists a $b \in B$ s.t. $a \leq b$.

Problem: Given A & B , decide whether $A \leq B$.

Naive solⁿ:

For every $a \in A$ linearly search B for a qualifying b

Size of $A, B = \underline{n}$.

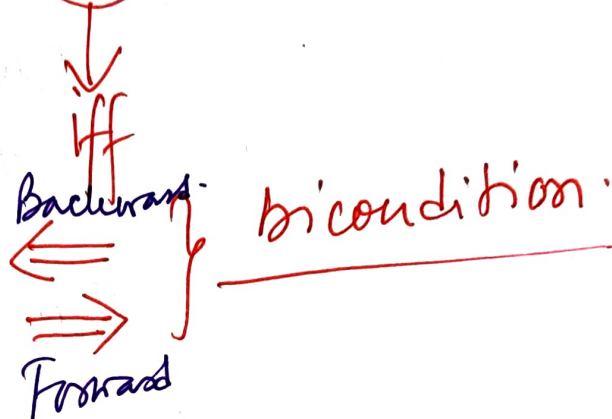
No. of operations

' n ' elements in A x up to ' n ' elements in B to be searched.

$O(n^2)$. $\rightarrow$ counting.

Can we do better?

Theorem: $A \leq B \iff \max(A) \leq \max(B)$.



$(\Rightarrow)$. Assume $A \leq B$

Let $m_A = \max(A)$. $m_A \in A$; also say $a = m_A$.
there exists $b \in B$ with $m_A \leq b$.

$\therefore b \leq \max(B) = m_B$, $\therefore m_A \leq m_B$. $\square$

$(\Leftarrow)$ Assume $\max(A) \leq \max(B)$.

Pick any $a \in A$. Then $a \leq \max(A)$.
(by defⁿ of $\max()$)

Combined with $\max(A) \leq \max(B)$,
~~we say~~ we can say $a \leq \max(B)$.

Choose $b \in B$ s.t. $b = \max(B)$

$\therefore a \leq b$. (true for all $a \in A$
because we started with an arbitrary a) $\square$

Efficiency : ~~One pass~~ One pass through A
to get the $\max(A)$ $[n]$

One pass through B.
to get the $\max(B)$ $[n]$

One comparison.

$$\begin{aligned}\text{Computations} &= 2n+1 \\ &= O(n).\end{aligned}$$

Define $A \rho B \Leftrightarrow \max(A) = \max(B)$.

Equivalence relation. $\left\{ \begin{array}{l} \text{Reflexive } a \rho a \\ \text{Symmetric } a \rho b \Rightarrow b \rho a. \\ \text{Transitive } a \rho b, b \rho c \Rightarrow a \rho c. \end{array} \right.$

$$= \{ n \in \mathbb{N} \mid \left. \begin{array}{l} a = a \text{ (R)} \\ a = b \Rightarrow b = a \text{ (S)} \\ a = b, b = c \Rightarrow a = c \text{ (T)} \end{array} \right\} \in E$$

$\emptyset: \max(A) = \max(B)$

E. $\left\{ \begin{array}{l} R: \max(A) = \max(A) \quad \checkmark \\ S: \max(A) = \max(B) \Rightarrow \max(B) = \max(A) \quad \checkmark \\ T: \max(A) = \max(B), \max(B) = \max(C) \\ \Rightarrow \max(A) = \max(C) \quad \checkmark \end{array} \right.$

Equivalence class / partition.

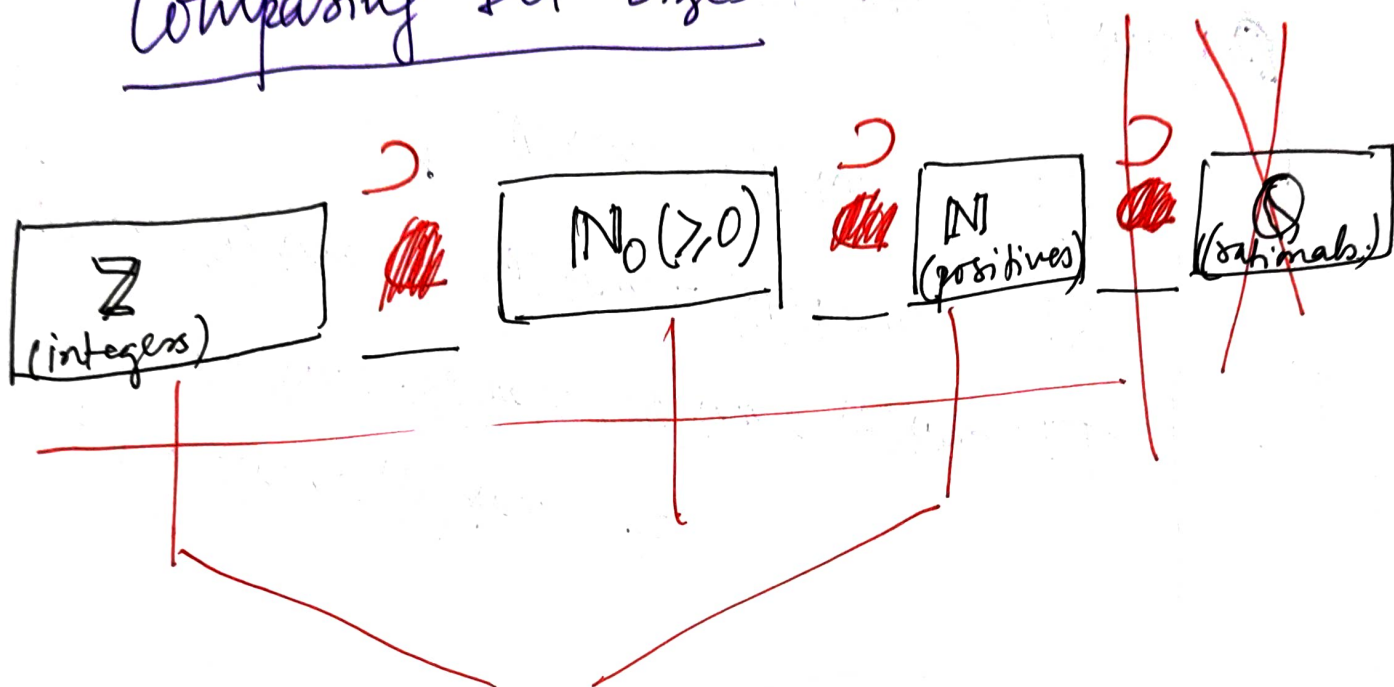
$\{A: \max(A) = 1\}, \{A: \max(A) = 2\}, \{A: \max(A) = 3\}, \dots$

How many such A?
 $\max([1]) = 1$
 $\max([1, 1, 1]) = 1$
 $\dots$
 infinite class.

$[1], [1, 2]$
 $[1, 2, 2],$
 $[2, 2, 2] \dots$
 infinite class

$\dots$
 infinite class.

Comparing set sizes



Sizes?

Same size $\leftarrow$

countably
infinite sets.

$\aleph_0$ (aleph-naught)

Bigger size sets exist?

$\mathbb{R}, \mathbb{C} \mapsto c$ (continuum)

$c > \aleph_0$

A computer can only solve countably many problems. ($\aleph_0$ many programs/algo exist).

The set of all computational problems is uncountable.

↳ There must exist a problem which non-computers will be able to solve.

↳ Alan Turing.

~~Counting:~~

Set A: $(2^{100})!$

Set B: $2^{100}!$

Which set is bigger? How do you say so?

Set A:

$$(2^{100})!$$

$$N = 2^{100}.$$

Compute $N!$

$$\log_2(N!) \approx N \log N$$

$$\hookrightarrow O(N \log N).$$

$$\log 1 \leq \log N$$

$$\log 2 \leq \log N$$

$$\log 3 \leq \log N$$

$$\vdots$$

$$\log 1 + \log 2 + \log 3 + \dots \leq N \log N$$

$$= O(N \log N).$$

$$N = 2^{100}$$

$$\log N = 100.$$

$$N \log N = 100 \times 2^{100}.$$

$$\text{Set B : } 2^{100!}$$

$$\log(2^{100!}) = 100!$$

$$100 \times 2^{100} \text{ vs } 100!$$

$$2^{100} \text{ with } 99!$$

$$99! = 1 \times 2 \times \dots \times 99.$$

$$= 1 \times 2 \times 3 \times 4 \times [5 \times 6 \times \dots \times 99].$$

$$\downarrow \quad \downarrow \quad \downarrow$$

$$> 2^2 > 2^2 \dots 2^2$$

$$99! > (2^2)^{95}$$

$$99! > 2^{190} \gg 2^{100}.$$

$$\text{Set B} \gg \text{Set A.}$$



$$abc.abc = abc \times \underbrace{1001}_{7 \times 11 \times 13}.$$


```

for i = 1 to n
  for j = 1 to i
    for k = 1 to j
      print("hello")

```

How many times will be "hello" printed?

↳ Counting problem?

Way to choose ordered triples $\langle i, j, k \rangle$ from $\{1, \dots, n\}$.

$$\text{Count} = \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1$$

$$= \sum_{i=1}^n \sum_{j=1}^i j$$

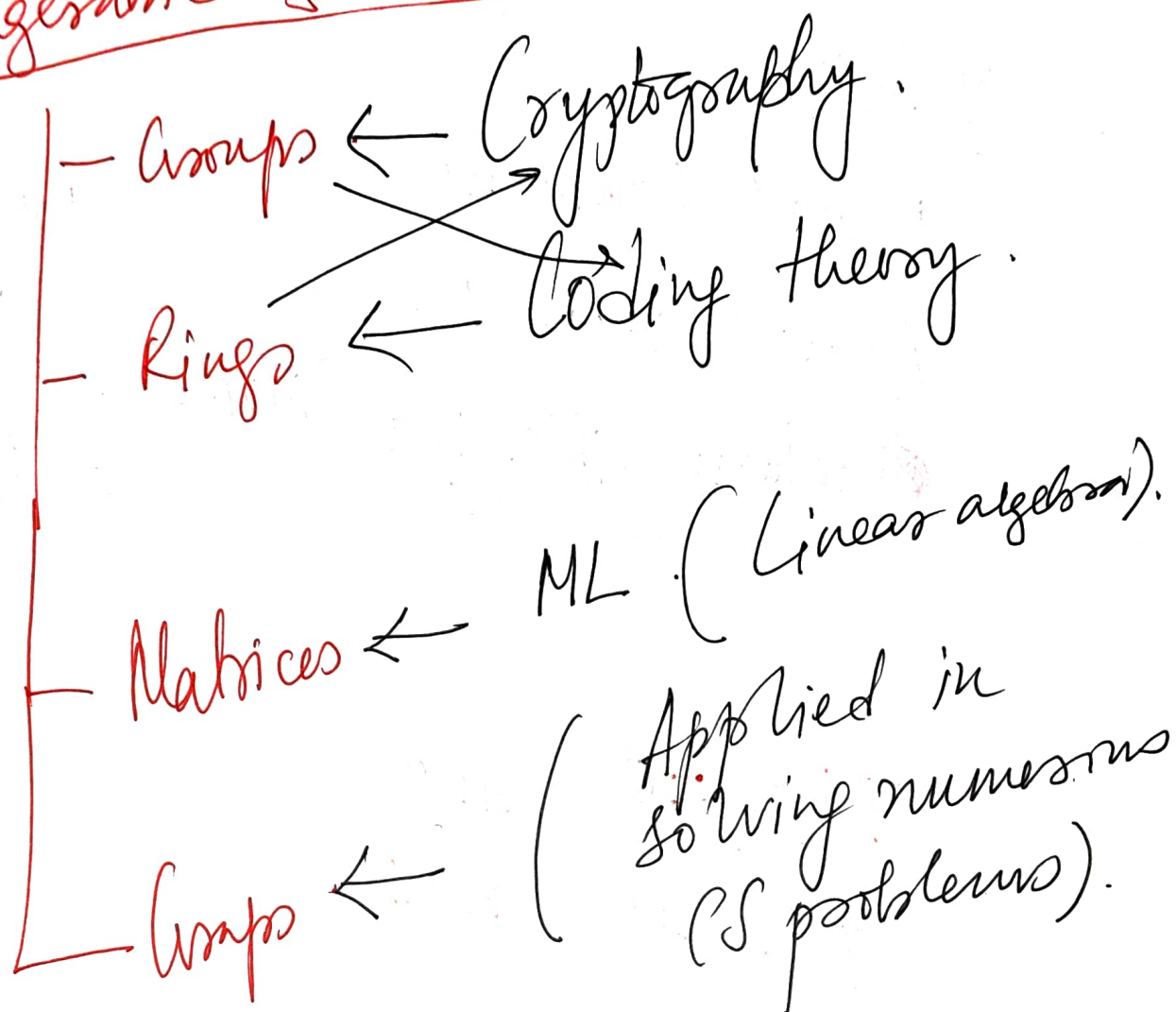
$$= \sum_{i=1}^n \frac{i(i+1)}{2}$$

$$= \sum_{i=1}^n \binom{i+1}{2} = \binom{n+2}{3}$$

↓
Hockey stick identity.

Choose $\binom{n+2}{3}$
C()

Algebraic Structures.



Continuous Maths
Limits, differentiability...
foundations → real numbers
 $\mathbb{R}$

Discrete maths.
Algebra, combinatorics
related structures.
Most natural
numbers.

→ CONCRETE MATHS.

fib(n).

if $n=0$: return 0

if $n=1$ return 1.

return fib(n-1) + fib(n-2).

Question

How many
operations
in terms of
 n ?

$$T(n) = 0 \quad n=0$$

$$T(n) = 0 \quad n=1.$$

$$T(n) = T(n-1) + T(n-2) + 1. \quad n > 1.$$

recurrence relation.